

Computable and uncomputable functions

Exercise sheet 1 (solutions)

Exercise 1. A total function $f: \mathbb{N} \rightarrow \mathbb{N}$ is called non-increasing if $f(n+1) \leq f(n)$ for all $n \in \mathbb{N}$. Prove that every non-increasing function is computable.

Proof. We first show that every non-increasing $f: \mathbb{N} \rightarrow \mathbb{N}$ is *eventually constant*: there exists a number N such that $f(x) = f(N)$ for every $x > N$. Intuitively, when x increases by 1, the value of f either stays the same or decreases, and it cannot decrease forever, since the values are natural numbers and cannot fall below 0. For the curious, we give a formal proof in the end.

Here is a program that computes f .

```
def compute_f(x):
    F = [f(0), ..., f(N)] # hardcoded array
    if x ≤ N:
        return F[x]
    else:
        return F[N]      # f(x) = f(N) for all x > N
```

We now prove that a non-increasing function is eventually constant by induction on $f(0)$.

Base case. If $f(0) = 0$, then, since f is non-increasing, $f(x) = 0$ for every x . Thus $N = 0$ works, and the claim holds.

Induction step. Suppose the claim holds for every non-increasing f with $f(0) \leq n$; we prove it for f with $f(0) = n + 1$. Two cases are possible.

- (1) f is constant, i.e. $f(x) = f(0)$ for every x . Then $N = 0$ works.
- (2) There exists $k > 0$ with $f(k) \neq f(0)$. Since f is non-increasing, $f(k) < f(0) = n + 1$, so $f(k) \leq n$. Consider $g(x) = f(x + k)$. The function g is non-increasing and total, and $g(0) = f(k) \leq n$; by the induction hypothesis, g is constant from some N' onward. Hence f is constant from $N = N' + k$ onward, and the claim holds. $\square$

Exercise 2. Let $f: \mathbb{N} \rightarrow \{0, 1\}$ be such that $f(n) = 1$ if the decimal expansion of π contains n consecutive 9s, and $f(n) = 0$ otherwise. Is f computable?

Solution. It is. We provide two proofs of this fact.

First proof. If the decimal expansion of π contains n consecutive 9s, then it contains m consecutive 9s for every $m < n$. Consequently, if $f(n) = 1$, then $f(m) = 1$ for every $m < n$, i.e. f is non-increasing. By the previous exercise, f is computable.

Second proof. Exactly one of the two following cases holds:

Case 1: π contains n consecutive 9s for arbitrarily large (and thus for every) $n \in \mathbb{N}$. Then f is the constant function $f(n) \equiv 1$, computed by the program:

```
def compute_f_case_1(n):  
    return 1
```

Case 2: there is a maximal number N of consecutive 9s occurring in π . That is, π contains n consecutive 9s if and only if $n \leq N$. By hardcoding N , we get the program:

```
def compute_f_case_2(n):  
    if n ≤ N:  
        return 1  
    return 0
```

□